PDF 中已有的层（核心业务分层）

根据你提供的 PDF，标准分层大致是：

层	职责
agents/	智能体核心：State、节点、路由、图编译、工具定义、记忆/HITL/多轮控制
api/	对外入口（HTTP / WebSocket 等）
models/	数据结构（State、请求/响应模型）
db/	数据访问（真实/模拟数据、数据库操作）
common/	配置、日志、通用工具
resources/ 或 static/	提示词、静态资源

这些已经覆盖了「智能体如何思考与行动」以及基础支撑，但在**企业级生产环境**还远远不够。

企业级完整分层补充（推荐结构）

在真实企业落地中，建议在原有基础上扩展为以下更完整的分层（按依赖方向从外到内）：

1. 接入与网关层（已有 api/ 的强化）

- **api/**（或 gateway/）：FastAPI / Fastify 等，支持 REST + SSE / WebSocket 流式
- 鉴权（JWT / OAuth / SSO）、限流、请求校验、版本控制（/v1）
- 任务路由、中断/取消接口、健康检查

2. 智能体编排层（已有 agents/ 的细化）

- **agents/** 或 **graphs/** + **nodes/** + **states/**
  - 主图、子图、多 Agent（Supervisor / Hierarchical / Swarm）
  - 节点工厂、条件边 / Command 路由
  - HITL（interrupt + resume）、迭代保护、结构化输出
- **tools/**（独立出来更清晰）：工具定义、权限校验、MCP 集成、沙箱执行
- **prompts/** 或 **resources/prompts/**：版本化提示词（最好支持 prompt registry）

3. 领域与业务逻辑层（常被忽略）

- **services/** 或 **domain/**
  - 纯业务规则、Business Logic Gate（不依赖 LangGraph 的校验逻辑）
  - 领域实体、策略（Policy）
- **middleware/**：Guardrails、脱敏、内容安全、成本控制、重试/熔断

4. 记忆与状态持久化层（db/ 的扩展）

- **memory/** 或强化 **db/**
  - 短期记忆（Redis）
  - 长期记忆 / 向量库（pgvector、Chroma 等）
  - Checkpointer（生产强烈推荐 PostgresSaver / RedisSaver，而不是 MemorySaver）
  - 会话 / Thread / Run 状态管理

5. 知识与检索层（RAG 相关）

- **rag/** 或 **knowledge/**
  - 文档加载、分块、Embedding、检索、重排序
  - 知识库连接器（企业知识源、SharePoint、内部 API 等）

6. 可观测与评估层（企业级刚需）

- **observability/**
  - 结构化日志、OpenTelemetry / LangSmith / Langfuse 追踪
  - 指标（Prometheus）、链路 trace_id / run_id
- **evals/**
  - 离线评测集、LLM-as-Judge、回归测试、A/B 测试
  - 质量门禁（上线前必须过）

7. 安全与治理层

- **security/** 或融入 middleware + control_plane
  - 输入/输出护栏、PII 脱敏、权限模型（细粒度工具/数据权限）
  - 审计日志、预算/成本上限、合规策略

- **auth/**：独立认证授权模块

8. 基础设施与运维层

- **common/** 或 **core/** + **infra/**
  - 配置（环境、Feature Flag）、日志、通用工具
  - LLM 工厂（多模型/多供应商切换、fallback）
  - 连接池、异步执行（Celery / 后台 Worker）
- **deployments/**、**docker/**、**k8s/**、**monitoring/**（Grafana 等）
- **tests/**、**scripts/**、**notebooks/**

9. 前端与交互层（可选但常见）

- 独立前端（React / 等）或嵌入式 UI
- 支持流式展示、HITL 审批界面、会话管理

推荐企业级目录骨架示例

```text
```

```
project/
├── api/                    # 接入层（FastAPI + 中间件链）
├── agents/                 # 或 graphs/ + nodes/ + states/
│   ├── graphs/
│   ├── nodes/
│   ├── states/
│   └── supervisors/        # 多 Agent 时
├── tools/                  # 工具定义与 MCP
├── prompts/                # 版本化提示词
├── services/               # 纯业务逻辑 / Domain
├── middleware/             # Guardrails、安全、成本
├── memory/                 # 短期 + 长期 + Checkpointer
├── rag/ 或 knowledge/      # 检索与知识
├── models/ 或 schemas/     # Pydantic / 数据模型
├── db/                     # 数据库访问
├── observability/          # 追踪、指标、日志
├── evals/                  # 评测与回归
├── security/ 或 auth/      # 鉴权与治理
├── common/ 或 core/        # 配置、LLM 工厂、工具函数
├── resources/              # 静态资源
├── tests/
├── deployments/            # Docker / K8s / Helm
├── langgraph.json          # 部署配置（若用 LangSmith Deployment）
└── pyproject.toml / requirements.txt
```

关键原则（企业级必守）

1. **依赖方向清晰**：上层依赖下层，agents 不主动控制 api/db；api 调用 agents，agents 使用 models/db/tools/common。
2. **横切能力下沉**：鉴权、限流、护栏、成本、观测放在 middleware / observability，而不是塞进 agents 节点。
3. **生产状态必持久化**：Checkpointer 用 Postgres / Redis，支持多实例、断点续跑、HITL。
4. **工具与业务分离**：工具权限校验、Business Gate 尽量纯函数，便于审计和测试。
5. **可观测 + 评测优先**：没有 tracing 和 evals，生产出问题几乎无法定位和回归。
6. **配置与代码分离**：模型选择、提示词版本、Feature Flag、限流策略外置。
7. **支持多 Agent 扩展**：从一开始预留 Supervisor / 子图结构，避免后期大重构。

最终一句话总结

PDF 中的层已经把「智能体大脑」和基础支撑讲清楚了。

企业级还需要补齐：

- **安全与治理**（鉴权、护栏、审计、成本）
- **记忆与持久化**（Checkpointer + 长短期记忆）
- **知识检索（RAG）**
- **可观测与评估（Observability + Evals）**
- **中间件与业务服务层**
- **部署与运维基础设施**

写代码时继续用 PDF 的判断口诀，同时多问一句：

「这是横切能力（安全/观测/记忆/评测）还是纯业务规则？」——是横切就放到对应专用层，不要全部塞进 agents/。

这样结构清晰、可测试、可审计、可扩展，才真正适合企业生产环境。